CPSC 490, Yale University
June Yoo
Advisor: Dr. Mike Shah

# A Federated Server Architecture for Real-Time Multiplayer Games

## Introduction

In networked multiplayer games, players must be able to read and write to some agreed game state. Such systems have strict and challenging performance requirements due to the real-time and write-heavy nature of games. Moreover, games are evolving from small intimate experiences to large communal ones, with recent “battle royale” and MMO (massively multiplayer online) games managing anywhere from hundreds to tens of thousands of players per instance. Finally, scalable virtual worlds which facilitate real-time interaction may become increasingly important outside of gaming as tech leaders look to bring virtual reality and the "metaverse" to the mainstream.

With this project, I aim to implement a protocol which can be used to implement a distributed server for real-time games, and a library/API which makes this protocol easily usable in the implementation of games. This would enable several new server deployment patterns currently uncommon in games: for instance, a federated network of regional servers could support a single, world-sized game instance which enables all players of the game to interact with each other in real time. Furthermore, players would also be able to host their own servers for large-scale games without straining or depending on a centralized maintainer. These use cases are partially inspired by the recent discussion around Bluesky and the AT Protocol for distributed social networks.

## Related Work

There has been limited research in the realm of distributing individual game instances so that a single world can be accessed by many players at once. These approaches to scaling games take advantage of the fact that most objects in games only need to be aware of their local surroundings to update themselves, rather than the entire map. *SimMud* [3] is one of the first such architectures to be completely decentralized, not needing a single authority to dictate the state of the game. In this model, the game world is partitioned into a large number of small geographical regions known as *cells*, each of which is then responsible for its own game state. This allows for a much larger number of concurrent accesses, with experimental results showing up to 4000 players in a single game. However, this approach requires connection handoff as objects move across regions, making it unsuitable for fast-paced gameplay such as that required by FPS (first-person shooter) games. Furthermore, SimMud is highly sensitive to *population density*, meaning that geographical regions in the game world that are activity hotspots ultimately become bottlenecks for the overall system.

*Colyseus* [2] improves on SimMud by decoupling the location of game objects in the game world from their server assignment. Colyseus' architecture implements both an object placer and an object locator that moves objects between servers and routes to them dynamically in a load-balanced fashion. Furthermore, it allows servers to hold *replicas* of other servers' objects to serve them closer to its clients, similar to a cache.

**Problem Description**

A handful of architectures dominate the online gaming landscape, all of which rely on some collection of authoritative single servers. In most popular multiplayer games, all players connect to a single endpoint owned by the publisher. These players are then load-balanced across *lobbies* or *zones*, which are separate instances of the game application. However, these instances are limited in capacity, do not communicate with each other in real time, and players are unable to migrate between lobbies without a significant overhead for connection handoff. [4] For instance, as of 2024, individual instances of Fortnite remained limited to a maximum of 100 players, despite supporting up to 3.4 million players across all of its instances. [1] These ceilings can be attributed to the inherent computational and bandwidth bottlenecks of a single-server architecture, suggesting that game servers need to be distributed across nodes to maximize system throughput.

Another approach, typically employed in single-player games with multiplayer features, allows players to host small worlds from their own machine and invite other players. This reduces the cost and complexity of hosting for the publisher, but is typically limited by the player's machine. [5]

Therefore, we want to implement a fast federated architecture which could potentially solve the shortcomings of both; for instance, publishers could scale single game worlds to hundreds of concurrent players while maintaining real-time gameplay. Furthermore, this tool would also allow small groups of players to host more powerful multiplayer experiences using their collective machines instead of relying on one host to support all of them.

Of course, this approach has its own shortcomings; in particular, this structure is not particularly robust against complete or Byzantine failures of a single node in the network. However, it is likely that this risk is tenable in gaming, as 1) any extended network failure is catastrophic in real-time games, and should be resolved at the source and 2) players hosting their own multiplayer experiences will likely hold each other accountable against modifications and cheating.

**Planned Approaches**

I plan to implement the core underlying protocol closely following the Colyseus architecture. Colyseus consists of 3 components; an object locator, which enables nodes to subscribe to certain regions of the game world and be notified when other nodes publish game objects there; a replica manager, which manages loosely synchronized replicas of game objects on each node, and an object placer, which reassigns primary objects to nodes using a load-balancing algorithm. In the original Colyseus paper, the object placer is not implemented, leaving all primary objects attached to their node of origin, and we will likely follow this approach. Moreover, Colyseus is dependent on a complex distributed hash table called Mercury, which we will also rely on in this project.

Once we have a working version of the basic protocol, I would like to implement several optimizations and security features on top of the original, and evaluate whether or not they are beneficial to the overall efficiency of the network. For instance, I may attempt to transition Colyseus to an entirely event-driven model, add a mechanism to verify consistency between server nodes using checksums, or swap out Mercury for a faster, lighter tool.

I previously completed a prototypal implementation of the above as part of my coursework. Hence, I would like to extend on this work by wrapping the above protocol into a neatly wrapped library, similar to Valve's GameNetworkingSockets library. In order to support practical game development using

this architecture, I will identify the most common networking needs and message types used in online games and support each of these using the protocol above. Broadly, this means that I would need to reimplement the protocol to use UDP, a necessary prerequisite for real-time games, and add back configurable TCP-like features such as message reliability and orderedness. It also means that I would add in built-in tools to abstract away this networking information, so that servers implemented using this library can know what is happening on other servers as if it were on its own machine without having to perform complex validation, interpolation, or extrapolation on the network data.

**Timeline**

- February 7th — review/refactor/rewrite existing code | proposal proposal presentation
- February 14th — connect to Mercury or research other alternatives for implementing the object locator
- February 28th — add optimizations, finalize underlying protocol details & implementation, sketch out architecture and components
- March 14th — refactor messaging between servers to use UDP, modelling, or possibly using, Valve's GameNetworkingSockets library
- March 28th — add support for multiple messaging modes/features, like reliability and orderedness | midterm progress presentation
- April 14th — test/measure efficiency with games, buffer time for any outstanding work or stretch goals
- April 21st — finish final report, project poster for submission
- May 1st — final submission

**List of Deliverables**

- Source code for a game networking library built around federation, as described above
- Source code used for any automated testing/evaluation
- Poster summarizing research findings
- Final report summarizing research findings
- 10 minute video presentation/demonstration of project as applied to a game
- Add proposal and final report to CPSC 490 website

**Future Work**

Future extensions to this project, which could be implemented as stretch goals in the final weeks, include:

- More exploration of an efficient object placer implementation, which does not exist in the Colyseus protocol or the scope of this project
- Built-in encryption and security
- Adding robustness for single-node failures
- Adding robustness for configuration changes (that is, nodes exit or are added)
- General usability features to make this a practical solution for implementing games

**References**

[1] Postmortem of Service Outage at 3.4M CCU. 2018.
https://www.fortnite.com/news/postmortem-of-service-outage-at-3-4m-ccu.

[2] Ashwin Bharambe, Jeffrey Pang, and Srinivasan Seshan. Colyseus: a distributed architecture for online multiplayer games. In Proceedings of the 3rd Conference on Networked Systems Design & Implementation - Volume 3, NSDI'06, page 12, USA, 2006. USENIX Association.

[3] B. Knutsson, Honghui Lu, Wei Xu, and B. Hopkins. Peer-to-peer support for massively multiplayer games. In IEEE INFOCOM 2004, volume 1, page 107, 2004.

[4] Kuan-Ta Chen, Polly Huang, Chun-Ying Huang, and Chin-Laung Lei. Game traffic analysis: an mmorpg perspective. In Proceedings of the International Workshop on Network and Operating Systems Support for Digital Audio and Video, NOSSDAV '05, page 19–24, New York, NY, USA, 2005. Association for Computing Machinery.

[5] Kaiwen Zhang and Bettina Kemme. Transaction models for massively multiplayer online games. 2011 IEEE 30th International Symposium on Reliable Distributed Systems, pages 31–40, 2011.